SOFTWARE REQUIREMENTS SPECIFICATION

G2 — Data & Intelligence Subsystem

Public Transport Real-Time Tracking & ETA Prediction
Sri Lanka — Moratuwa → Kadawatha(like) Express Bus Route

Project	Group G — Public Transport System
Subgroup	G2 — Data & Intelligence
Document Version	1.0 (Base)
Date	April 7, 2026
Status	Draft

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, Acronyms, and Abbreviations

1.4 References

1.5 Document Overview

2. Overall System Description

2.1 System Context

2.2 Product Perspective

2.3 Product Functions

2.4 User Classes and Characteristics

2.5 Operating Environment

2.6 Design and Implementation Constraints

2.7 Assumptions and Dependencies

3. Agile User Stories & Acceptance Criteria

3.1 User Story 1: Live Tracking & ETA

3.2 User Story 2: Occupancy Level

3.3 User Story 3: Route Discovery

3.4 User Story 4: Anomaly Alerting (G2 Specific)

4. Functional Requirements

4.1 Data Ingestion

4.2 Stream Processing & Feature Engineering

4.3 ETA Prediction Model

4.4 Anomaly Detection

4.5 REST API & WebSocket

4.6 Geofencing & Boarding Logic

5. Non-Functional Requirements

5.1 Performance

5.2 Scalability

5.3 Reliability

5.4 Security

5.5 Maintainability

6. External Interface Requirements

6.1 G1 Interface (Input)

6.2 G3 Interface (Output)

6.3 G4 Interface (Deployment)

7. Data Requirements

7.1 GPS Reading Schema

7.2 ML Feature Set
7.3 ETA Response Schema
7.4 Anomaly Alert Schema
8. System Architecture
8.1 Component Overview
8.2 ML Model Architecture
8.3 Database Design
9. Constraints and Edge Cases
10. Appendix — API Endpoint Summary

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete set of functional and non-functional requirements for the G2 — Data & Intelligence subsystem of the Group G Public Transport System. This document serves as the authoritative reference for all design, implementation, testing, and integration activities performed by G2.

1.2 Scope

The G2 subsystem is responsible for ingesting real-time GPS data from edge devices (G1), processing it through ML-powered pipelines, providing ETA predictions and anomaly detection, and exposing results via a REST and WebSocket API to the G3 user interface.

The initial operational scope covers one route:

- Route: Moratuwa Bus Stand → Kadawatha Bus Stand
- Segment A (Urban): Moratuwa → Kahathuduwa Entrance (~18 km)
- Segment B (Highway): Kahathuduwa → Gelanigama Exit (~45 km at 80-100 km/h)
- Segment C (Urban): Gelanigama → Kadawatha (~8 km)

1.3 Definitions, Acronyms, and Abbreviations

Abbreviation	Meaning
ETA	Estimated Time of Arrival
GPS	Global Positioning System
MQTT	Message Queuing Telemetry Transport (IoT messaging protocol used by G1)
Kafka	Apache Kafka — distributed event streaming platform
Flink	Apache Flink — distributed stream processing framework
PostGIS	Spatial extension for PostgreSQL enabling geographic queries
ML	Machine Learning
XGBoost	Extreme Gradient Boosting — the primary ETA regression algorithm
API	Application Programming Interface
REST	Representational State Transfer
NFR	Non-Functional Requirement
FR	Functional Requirement
G1/G2/G3/G4	Subgroups of Group G as defined in the project brief
Geofence	A virtual geographic boundary defined by GPS coordinates
WS	WebSocket — full-duplex communication protocol

1.4 References

- Group G Project Brief — Public Transport System (April 2026)
- Requirement Engineering — Problem Identification document (Group G, April 2026)
- G2 Architecture Design Document (docs/architecture.md)
- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications

- Apache Kafka Documentation — https://kafka.apache.org/documentation/
- Apache Flink Documentation — https://flink.apache.org/
- FastAPI Documentation — https://fastapi.tiangolo.com/
- PostGIS Documentation — https://postgis.net/documentation/
- XGBoost Documentation — https://xgboost.readthedocs.io/

1.5 Document Overview

Section 2 provides the overall system context. Section 3 defines user stories and acceptance criteria. Sections 4-5 specify functional and non-functional requirements. Section 6 defines inter-group interfaces. Sections 7-8 cover data requirements and architecture. Section 9 lists constraints and edge cases.

2. Overall System Description

2.1 System Context

G2 is the central intelligence layer of the four-subgroup architecture. It sits between the hardware edge (G1) and the user-facing applications (G3), with infrastructure managed by G4. G2 has no direct user interface — it is a backend data service consumed entirely through its API.

2.2 Product Perspective

The system addresses the problem of commuters at intermediate bus halts (e.g., Katubedda, Rawathawatta) having no reliable information about approaching express buses. Without arrival time data, commuters over-wait or miss buses entirely.

G2 solves this by:

- Processing real-time GPS from buses at 1 Hz
- Predicting per-stop arrival times using historical traffic patterns and current speed
- Detecting delays, route deviations, and service anomalies in real-time
- Exposing clean, low-latency data to the G3 user interface

2.3 Product Functions

Function	Description
GPS Stream Processing	Ingest, clean, and enrich 1 Hz GPS data from all active buses
Feature Engineering	Extract 16 ML-ready features per GPS point in real-time
ETA Prediction	Predict arrival time at each downstream stop with confidence score
Anomaly Detection	Detect and classify service anomalies (speed, route, dwell)
Geofencing	Enforce boarding cutoff at Kahathuduwa highway entrance
REST API	Expose ETA and anomaly data to G3 via HTTP endpoints
Live WebSocket Feed	Push fleet status to G3 every second
Historical Storage	Persist trips and GPS readings for model retraining

2.4 User Classes and Characteristics

Actor	Role	Auth Needed
Passenger / Commuter	Views ETA and bus location via G3 app. No direct interaction with G2.	None
Ops Dashboard User	Monitors fleet health, anomalies, and KPIs via G3 dashboard.	None
Driver	Receives route deviation alerts and notifications via G3 driver display.	None
G1 Edge Device	Transmits GPS readings over MQTT. Machine-to-machine interaction.	API Key

G3 Frontend	Consumes REST and WebSocket API. Developer-facing integration.	API Key
G4 Platform	Deploys and monitors G2 containers. Accesses /health and /metrics.	None

2.5 Operating Environment

- Python 3.12 runtime
- Docker containers orchestrated by G4 (Kubernetes in production)
- Apache Kafka 7.6 (Confluent) for message streams
- Apache Flink (PyFlink) for stream processing jobs
- PostgreSQL 16 + PostGIS 3.4 for storage and spatial queries
- Redis for API response caching
- Linux server environment (Ubuntu 22.04 LTS)

2.6 Design and Implementation Constraints

- CON-1: All services must be containerised via Docker and conform to G4 deployment standards.
- CON-2: Inter-subgroup communication follows defined interface contracts (MQTT topics and REST schemas).
- CON-3: No hardcoded secrets; all configuration via environment variables.
- CON-4: GPS occupancy data is simulated or conductor-input for the MVP; hardware sensor integration is out of scope.
- CON-5: The system is scoped to the Moratuwa→Kadawatha route for Deliverable 1.
- CON-6: All GPS coordinates must be validated within Sri Lanka's bounding box (lat 5.9–9.9°N, lng 79.5–81.9°E).

2.7 Assumptions and Dependencies

- G1 provides GPS data at 1 Hz per active bus in the agreed MQTT JSON format.
- G4 provides a running Kafka and PostgreSQL infrastructure accessible by G2 services.
- Historical trip data for model training is either synthetically generated or sourced from the GPS simulator.
- The Kahathuduwa highway geofence polygon is fixed and agreed upon by all sub-teams.
- G3 handles all user authentication; G2 only authenticates machine-to-machine callers.

3. Agile User Stories & Acceptance Criteria

These user stories were derived from the RE process (Problem Identification document). G2 is not a user-facing system but is the enabling layer that makes each story technically possible. Acceptance criteria are stated from the end-user's perspective and decomposed into G2 engineering tasks below.

3.1 User Story 1: Live Tracking & ETA (High Priority)

Story: As a commuter waiting at an intermediate halt (e.g., Katubedda), I want to view the real-time location and ETA of the next Kadawatha-bound express bus, so I can time my arrival at the halt.

Acceptance Criteria:

- The app displays the bus moving on a map with less than a 5-second delay. [G2 obligation: deliver GPS-derived bus positions via WebSocket /live-feed within 1 second of receipt.]
- The ETA updates dynamically based on current traffic before the Kahathuduwa entrance. [G2 obligation: ETA model refreshes every Flink window cycle (~5s) and is published to the API.]

3.2 User Story 2: Occupancy Level (Medium Priority)

Story: As a commuter, I want to see the current occupancy level (Seats Available / Standing Only / Full), so I can decide if I should wait for this bus or find an alternative.

Acceptance Criteria:

- The app displays a visual indicator (Green/Yellow/Red) of crowd levels. [G2 obligation: include occupancy field in live-feed payload; initial MVP uses simulated values.]
- Buses marked 'Full' automatically notify users at upcoming stops. [G2 obligation: anomaly/alert engine emits occupancy-full event consumed by G3 notification service.]

3.3 User Story 3: Route Discovery (Low Priority)

Story: As a new user, I want to view the complete path of the Moratuwa–Kadawatha route including all valid stops before the highway, so I know where I can catch the bus.

Acceptance Criteria:

- Route line and all stops displayed on map. [G2 obligation: serve route GeoJSON from GET /routes/{route_id}.]
- Stops past Kahathuduwa marked as boarding-unavailable. [G2 obligation: is_highway_entry flag in stops schema.]

3.4 User Story 4: Anomaly Alerting (G2 Specific)

Story: As an operations controller, I want to receive real-time alerts when a bus deviates from its route, experiences unusual delays, or loses GPS signal, so I can take corrective action.

Acceptance Criteria:

- Alert appears within 5 seconds of anomaly onset. [G2 obligation: rule engine fires within current Flink window.]
- Each alert includes severity (warning/critical) and a recommended action.

- Alerts for resolved anomalies are cleared from the active list.

4. Functional Requirements

4.1 Data Ingestion

ID	Requirement	Priority
FR-1.1	The system shall consume GPS messages from the Kafka topic gps.raw.{bus_id} published by the MQTT-to-Kafka bridge.	Must Have
FR-1.2	The MQTT bridge shall subscribe to MQTT topics gps/bus/+ and produce validated JSON to the corresponding Kafka topic at the received frequency (1 Hz).	Must Have
FR-1.3	The system shall reject and route to a dead letter topic (gps.dlq) any GPS message that fails Pydantic schema validation or contains coordinates outside the Sri Lanka bounding box.	Must Have
FR-1.4	The system shall persist raw and cleaned GPS readings to the PostgreSQL gps_readings table partitioned by date for historical analysis.	Must Have

4.2 Stream Processing & Feature Engineering

ID	Requirement	Priority
FR-2.1	The system shall apply a Kalman filter to each incoming GPS point to reduce positional noise before feature extraction.	Must Have
FR-2.2	The system shall map-match each cleaned GPS point to the nearest segment of the predefined route geometry.	Must Have
FR-2.3	The system shall extract exactly 16 ML features from each GPS data point as defined in Section 7.2 of this document.	Must Have
FR-2.4	The Apache Flink pipeline shall use a tumbling window of 5 seconds and a watermark allowance of 3 seconds for late-arriving GPS messages.	Must Have
FR-2.5	Enriched feature vectors shall be published to the Kafka topic gps.features for downstream ML model consumption.	Must Have

4.3 ETA Prediction Model

ID	Requirement	Priority

FR-3.1	The ETA model shall predict the remaining travel time in seconds to each downstream stop for a given bus.	Must Have
FR-3.2	The system shall use an XGBoost regressor as its primary ETA prediction algorithm.	Must Have
FR-3.3	When the trained XGBoost model is unavailable or its confidence falls below 0.5, the system shall fall back to a physics-based heuristic: ETA = distance_remaining / effective_speed.	Must Have
FR-3.4	The ETA model shall switch from the urban-traffic model variant to the highway-cruise model variant when the bus GPS position crosses the Kahathuduwa geofence polygon.	Must Have
FR-3.5	ETA predictions shall be refreshed at every Flink processing cycle (approximately every 5 seconds).	Must Have
FR-3.6	The system shall support batch retraining of the ETA model using accumulated historical trip data via the train_models.py script.	Should Have

4.4 Anomaly Detection

ID	Requirement	Priority
FR-4.1	Layer 1 (Statistical): The system shall compute a rolling Z-score over speed, dwell_time, and heading_change in a 5-minute window; a |Z| > 2.5 generates a warning; |Z| > 3.5 generates a critical alert.	Must Have
FR-4.2	Layer 2 (ML): The system shall run an Isolation Forest on the 16-feature vector continuously; anomaly scores below the contamination threshold generate a warning.	Should Have
FR-4.3	Layer 3 (Rules): The system shall evaluate the following rule-based conditions for every GPS update: (a) off-route deviation >200m for >30s, (b) stopped >5 min at non-stop location, (c) no GPS for >60s, (d) speed >120km/h on highway or >60km/h on urban segment, (e) bus exits route corridor polygon.	Must Have
FR-4.4	The result aggregator shall consolidate outputs from all three	Must Have

	detection layers into a single anomaly record with a unified confidence score.	
FR-4.5	Anomaly records shall be stored in the PostgreSQL anomalies table and published to the Kafka topic alerts.anomaly.	Must Have
FR-4.6	The system shall mark anomaly records as resolved when the triggering condition is no longer observed.	Should Have

4.5 REST API & WebSocket

ID	Requirement	Priority
FR-5.1	The system shall expose GET /eta/{bus_id} returning per-stop ETA predictions for the specified active bus.	Must Have
FR-5.2	The system shall expose GET /eta/{bus_id}/{stop_id} returning ETA for a specific stop.	Must Have
FR-5.3	The system shall expose GET /anomalies/{bus_id} returning the last 10 anomaly records for a bus.	Must Have
FR-5.4	The system shall expose GET /anomalies/active returning all currently unresolved anomalies across the fleet.	Must Have
FR-5.5	The system shall expose POST /ingest/gps for receiving GPS data directly (bypass Kafka for testing).	Should Have
FR-5.6	The system shall expose GET /routes/{route_id}/buses listing all active buses on a route.	Must Have
FR-5.7	The system shall expose WS /live-feed and push a full fleet status payload to all connected WebSocket clients every 1 second.	Must Have
FR-5.8	The system shall expose GET /health returning service status including database and Kafka connectivity.	Must Have
FR-5.9	The system shall expose GET /metrics in Prometheus format for G4 monitoring.	Must Have

4.6 Geofencing & Boarding Logic

ID	Requirement	Priority
FR-6.1	The system shall define and persist the Kahathuduwa highway entrance geofence polygon in the PostGIS geofences table.	Must Have

FR-6.2	When a bus GPS position is contained within the Kahathuduwa geofence polygon, the system shall set that bus's status field to 'boarding_unavailable' in all API responses.	Must Have
FR-6.3	The system shall reset a bus's boarding status to 'active' when its position exits the geofence (e.g., beyond Gelanigama exit).	Must Have
FR-6.4	The system shall expose spatial queries via PostGIS to allow G3 to ask 'which buses are currently between Stop A and Stop B'.	Should Have

5. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	End-to-end latency from GPS capture (G1) to API response (G3) shall not exceed 2.0 seconds under normal load.
NFR-2	Performance	The FastAPI server shall respond to REST endpoint requests with a p95 latency below 100 ms.
NFR-3	Performance	The WebSocket live-feed shall push updates to connected clients at 1-second intervals with a jitter of < 200 ms.
NFR-4	Performance	Bus positions on the G3 map shall not lag more than 5 seconds behind real-world position.
NFR-5	Scalability	The Kafka consumer architecture shall support scaling horizontally to process up to 50 concurrent buses without re-architecture.
NFR-6	Scalability	The FastAPI server shall be stateless so that G4 can horizontal-scale via Docker replicas.
NFR-7	Reliability	The MQTT-to-Kafka bridge shall implement exponential backoff reconnection and must resume within 10 seconds of MQTT broker recovery.
NFR-8	Reliability	GPS gaps of up to 30 seconds shall be handled gracefully using the last known speed and historical segment speed for ETA.
NFR-9	Reliability	The system shall achieve 99% uptime during operational hours (05:00–23:00 local time).
NFR-10	Security	All GPS ingestion endpoints (POST /ingest/gps) shall require a valid X-API-Key header. Invalid keys return HTTP 401.
NFR-11	Security	Public REST endpoints shall be rate-limited to 60 requests per minute per source IP. Excess returns HTTP 429.
NFR-12	Security	All database queries shall use parameterised statements (ORM) to prevent SQL injection.
NFR-13	Security	No credentials, API keys, or secrets shall be hardcoded; all are loaded from environment variables.
NFR-14	Maintainability	All Python functions shall include type hints. All data I/O shall use Pydantic models.

NFR-15	Maintainability	Code coverage (via pytest) shall not drop below 70% for model and API modules before merge.
NFR-16	Portability	All services shall run in Docker containers and must not have hard dependencies on the host OS.

6. External Interface Requirements

6.1 G1 Interface — Input

G1 (Device & Edge Systems) publishes GPS data via MQTT. G2 consumes it.

Property	Value
Protocol	MQTT over TCP (port 1883), bridged to Apache Kafka by G2's mqtt_bridge.py
MQTT Topics	gps/bus/{bus_id} — position data gps/bus/{bus_id}/status — bus status events
Payload Format	{"bus_id": "BUS-001", "lat": 6.7736, "lng": 79.8820, "speed": 35.2, "heading": 45.0, "timestamp": "2026-04-07T08:30:00Z", "route_id": "MOR-KAD-01"}
Frequency	1 Hz per active bus (one message per second)
Authentication	MQTT connect credentials managed by G4; G2 bridge receives as subscriber
Contract Owner	Agreed jointly between G1 and G2; G1 must not change field names without notice

6.2 G3 Interface — Output

G3 (Sys Eng & Interaction) consumes G2's API.

Property	Value
Base URL	http://g2-api:8000 (internal Docker network) or via G4 API gateway
REST Format	JSON, UTF-8, Content-Type: application/json
WebSocket URL	ws://g2-api:8000/live-feed
WS Push Rate	1-second intervals; payload: array of all active buses with position, speed, ETA, status
Authentication	X-API-Key header for server-to-server calls; public endpoints are open but rate-limited
CORS	Origins permitted: G3 frontend URLs, configurable via environment variable
Versioning	URL prefix /v1/; backward-compatible changes do not increment version

6.3 G4 Interface — Deployment & Monitoring

Property	Value
Docker Image	Dockerfile at docker/Dockerfile; built and tagged by G4 CI/CD pipeline
Health Endpoint	GET /health — returns 200 OK with JSON status of all dependencies
Metrics Endpoint	GET /metrics — Prometheus text format; scraped by G4 Prometheus instance
Key Metrics	gps_messages_received_total, eta_predictions_total, anomalies_detected_total, api_request_duration_seconds
Environment Config	All config via environment variables; template in docker/.env.example
Docker Compose	docker/docker-compose.yml provides local dev stack (Kafka, Zookeeper, Postgres, Redis, App)

7. Data Requirements

7.1 GPS Reading Schema

Field	Type	Example	Notes
bus_id	string	BUS-001	Unique bus identifier; matches G1 device ID
lat	float (WGS84)	6.7736	Latitude; valid range 5.9–9.9
lng	float (WGS84)	79.8820	Longitude; valid range 79.5–81.9
speed	float (km/h)	35.2	Instantaneous speed reported by G1 device
heading	float (°)	45.0	Compass bearing 0–360
timestamp	ISO 8601 UTC	2026-04-07T08:30:00Z	GPS measurement time
route_id	string	MOR-KAD-01	Assigned route identifier

7.2 ML Feature Set (16 Features)

#	Feature Name	Type	Description
1	hour_of_day	int	0–23
2	minute_of_hour	int	0–59
3	day_of_week	int	0=Mon, 6=Sun
4	is_weekend	bool	True on Saturday/Sunday
5	is_peak_hour	bool	06:30–09:00 and 16:30–19:30 local time
6	current_speed	float	km/h at this GPS point
7	avg_speed_5min	float	Mean speed over 5-minute rolling window
8	speed_variance_5min	float	Speed variance over 5-minute rolling window
9	distance_to_next_stop	float (m)	Great-circle distance to next scheduled stop
10	fraction_route_completed	float	0.0 at origin, 1.0 at destination
11	current_segment_id	int	1=Urban-A, 2=Highway-B, 3=Urban-C
12	scheduled_delay_seconds	float	Actual elapsed time minus scheduled elapsed time
13	time_since_departure	float (s)	Seconds elapsed since trip start
14	heading	float (°)	Current compass bearing
15	heading_variance_5min	float	Variance of heading over 5-minute window
16	distance_from_route_center	float (m)	Perpendicular distance from ideal route line

7.3 ETA Prediction Response Schema

Field	Type	Example	Notes
bus_id	string	BUS-001	Bus identifier

predictions	array	[...]	List of per-stop ETA objects
predictions[].stop_id	string	KATUBEDDA	Stop identifier
predictions[].eta_seconds	int	180	Seconds until predicted arrival
predictions[].confidence	float	0.85	Model confidence 0.0–1.0
model_version	string	v1.2	Model artifact version used
timestamp	ISO 8601	2026-04-07T08:30:00Z	Time of prediction

7.4 Anomaly Alert Schema

Field	Type	Example	Notes
bus_id	string	BUS-001	Bus identifier
anomaly_type	string	speed_spike	One of: speed_spike, long_dwell, signal_gap, off_route, speed_violation, geofence_exit
severity	string	warning	"warning" or "critical"
confidence	float	0.87	Detection confidence 0.0–1.0
details	object	{"z_score": 3.2}	Type-specific metadata (speeds, z-scores, distances)
recommended_action	string	Notify driver	Human-readable suggested response
location	object	{"lat": 6.74, "lng": 80.02}	WGS84 coordinates of anomaly
timestamp	ISO 8601	2026-04-07T08:35:00Z	Detection time
resolved	bool	false	True when condition clears

8. System Architecture

8.1 Component Overview

Module	Location	Responsibility
mqtt_bridge.py	streaming/	Bridges G1 MQTT topics to Kafka; validates payload
kafka_consumer.py	streaming/	Consumes cleaned GPS features; triggers ML inference
flink_pipeline.py	streaming/	PyFlink job: clean → features → predict → detect
gps_cleaner.py	utils/	Kalman filter, bounding-box check, map matching
features.py	utils/	Extracts all 16 ML features from a GPS point + context
geo.py	utils/	PostGIS helpers, geofence intersection, great-circle distance
eta_model.py	models/	XGBoost ETA regressor + physics fallback + mode switch
anomaly_model.py	models/	3-layer anomaly detection with result aggregation
route_validator.py	models/	Geofence checks, boarding status management
main.py	app/	FastAPI server: mounts all routers, configures CORS and middleware
schemas.py	app/	Pydantic v2 request/response models for all endpoints
database.py	app/db/	Async SQLAlchemy engine + PostGIS connection pool

8.2 ML Model Architecture

Model	Algorithm	I/O	Notes
ETA Regressor (Primary)	XGBoost	16 features → remaining_seconds	2× variants: urban + highway; trained independently
ETA Fallback	Physics Heuristic	distance, speed → remaining_seconds	distance_remaining / effective_speed; no training needed
Anomaly Layer 1	Z-Score (statistical)	Rolling 5-min window → Z score	Per-feature thresholds: warn >2.5, critical >3.5
Anomaly Layer 2	Isolation Forest	16 features → anomaly score	Trained on normal trips; contamination=0.05
Anomaly Layer 3	Rule Engine	GPS + context → pass/fail rules	5 hard rules; fires on single GPS point
Result Aggregator	Weighted voting	3-layer outputs → unified alert	Confidence = weighted avg of contributing detector scores

8.3 Database Design — Key Tables

Table	Purpose
routes	Route definitions with PostGIS LINESTRING geometry for the route line

stops	Individual stops along a route with POINT geometry, sequence order, and highway flags
buses	Registered bus fleet with assigned route and current status
gps_readings	All GPS points (raw + cleaned) partitioned by date; PostGIS POINT indexed for spatial queries
trips	Completed or in-progress journeys; used as training data for ML models
stop_arrivals	Per-stop actual vs. scheduled arrival times; primary source for delay analysis
anomalies	Timestamped anomaly records with PostGIS location, severity, type, confidence, and resolved flag
geofences	Named polygon regions (highway entry/exit, boarding cutoff) as PostGIS POLYGON

9. Constraints and Edge Cases

9.1 Highway Mode Switch

The most critical domain-specific constraint: when a bus enters the Southern Expressway at Kahathuduwa, the ETA model MUST switch from the urban-traffic model to the highway-cruise model. Failure to switch will cause severe over-estimation of ETA (urban model expects frequent slow-downs that do not occur on the expressway).

- Trigger: ST_Contains(kahathuduwa_polygon, bus_current_position) = TRUE
- Action: Set model_variant = 'highway'; update bus status to 'boarding_unavailable'
- Revert: When bus passes Gelanigama exit polygon, switch back to urban model

9.2 GPS Signal Loss

Buses may pass through tunnels or encounter device failures.

- Gap < 30s: Use last known speed + historical segment speed for ETA interpolation
- Gap 30–60s: Reduce ETA confidence to 0.3; include uncertainty in response
- Gap > 60s: Set bus status to 'unknown'; fire signal_gap anomaly (critical)

9.3 Cold Start (No Trained Model)

- On first deployment with no trained model: physics heuristic is used exclusively
- API response includes model_version: 'heuristic' to signal fallback mode to G3
- Model training is triggered manually via scripts/train_models.py once data exists

9.4 Occupancy Data (MVP Limitation)

Hardware occupancy sensors are out of scope for this deliverable. The system will use simulated or conductor-input occupancy levels. This decision is documented as a system constraint and the occupancy field is designed to accept real sensor data in future releases.

9.5 Single Route MVP

The system is architected for multi-route operation via the route_id field, but only the Moratuwa→Kadawatha route is operationally configured in Deliverable 1. Additional routes require only data seeding, not code changes.

10. Appendix — API Endpoint Summary

Method	Path	Parameters	Response	Consumer
GET	/health	None	200 OK + dependency status	All callers
GET	/api/v1/eta/{bus_id}	None	ETA predictions for all downstream stops	G3
GET	/api/v1/eta/{bus_id}/{stop_id}	None	ETA to a specific stop	G3
GET	/api/v1/anomalies/{bus_id}	?limit=10	Last N anomalies for a bus	G3
GET	/api/v1/anomalies/active	None	All current unresolved anomalies	G3 (Ops)
POST	/api/v1/ingest/gps	GPS JSON body	201 Created (test endpoint)	G1 / Test
GET	/api/v1/routes	None	List of available routes with stops	G3
GET	/api/v1/routes/{route_id}/buses	None	Active buses on a route	G3
GET	/metrics	None	Prometheus metrics	G4
WS	/ws/live-feed	None	1-second fleet status push stream	G3

Example WebSocket message payload pushed every 1 second to /ws/live-feed:

```
{
  "timestamp": "2026-04-07T08:30:01Z",
  "buses": [
    {
      "bus_id": "BUS-001",
      "lat": 6.7960,
      "lng": 79.9010,
      "speed": 38.5,
      "heading": 42.1,
      "route_id": "MOR-KAD-01",
      "status": "active",
      "eta_next_stop": 95,
      "next_stop_id": "RAWATHAWATTA",
      "occupancy": "available",
      "active_anomalies": 0
    }
  ]
}
```